

Back-end em python



Prof^a. Ma. Ana Clara

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 26 - Padrões de Criação: Factory e Singleton



Objetivos da Aula

- Compreender os padrões de criação de objetos
- Estudar e aplicar Factory Method e Singleton em Python
- Desenvolver exemplos práticos, incluindo implementação de um Logger
- Preparar os alunos para usar padrões em sistemas reais



Aula 26 - Padrões de Criação: Factory e Singleton

O que são Padrões de Criação?

- Lidam com a **instanciação de objetos**
- Garantem flexibilidade na criação de classes
- Evitam dependências rígidas no código





Aula 26 - Padrões de Criação: Factory e Singleton

Benefícios dos Padrões de Criação

- Centralizam lógica de criação
- Diminuem acoplamento
- Facilitam testes e manutenção





Aula 26 - Padrões de Criação: Factory e Singleton

Introdução ao Factory Method

- Cria objetos sem expor a lógica de criação
- Permite que subclasses decidam qual objeto instanciar





Aula 26 - Padrões de Criação: Factory e Singleton

Estrutura do Factory Method

- Classe abstrata define método fábrica
- Subclasses implementam a criação do objeto





Aula 26 - Padrões de Criação: Factory e Singleton



Exemplo simples de Factory em Python

```
class Transporte:
    def entregar(self):
        pass

class Caminhao(Transporte):
    def entregar(self):
        return "Entrega por caminhão"

class TransporteFactory:
    def criar(self, tipo):
        if tipo == "caminhao":
            return Caminhao()
```



Aula 26 - Padrões de Criação: Factory e Singleton

Exercício - Implementando Factory

Criar uma fábrica de meios de pagamentos: Cartão, Pix, Boletto.

Retornar objeto correto





Aula 26 - Padrões de Criação: Factory e Singleton

Benefício do Factory Method

- Flexibilidade na criação de instâncias
- Fácil de adicionar novos tipos
- Reduz repetição de código





Aula 26 - Padrões de Criação: Factory e Singleton



Introdução ao Singleton

- Garante que uma classe tenha **apenas uma instância**
- Fornece um ponto global de acesso a ela



Aula 26 - Padrões de Criação: Factory e Singleton

Problema resolvido pelo Singleton

Situações em que **apenas uma instância** faz sentido:

- Conexão com banco de dados
- Configuração global
- Logger (registro de eventos)





Aula 26 - Padrões de Criação: Factory e Singleton

Estrutura do Singleton

- Construtor privado ou controlado
- Acesso por método estático
- Reuso da mesma instância





Aula 26 - Padrões de Criação: Factory e Singleton

Singleton em Python (básico)

```
class Singleton:
    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
        return cls._instancia
```





Aula 26 - Padrões de Criação: Factory e Singleton



Exemplo Singleton - Logger

```
class Logger:
    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia.logs = []
        return cls._instancia

    def registrar(self, mensagem):
        self.logs.append(mensagem)
```



Aula 26 - Padrões de Criação: Factory e Singleton



Uso do Logger Singleton

```
log1 = Logger()
log2 = Logger()
log1.registrar("Primeira mensagem")
log2.registrar("Segunda mensagem")
print(log1.logs) # ['Primeira mensagem', 'Segunda mensagem']
```

Ambos objetos compartilham o mesmo estado



Aula 26 - Padrões de Criação: Factory e Singleton



Exercício - Singleton Configuração

Criar classe de Configuração Singleton. Armazenar parâmetros como host, porta e debug.



Aula 26 - Padrões de Criação: Factory e Singleton

Vantagens do Singleton

- Garantia de instância única
- Controle centralizado
- Útil para logging, conexões e configurações





Aula 26 - Padrões de Criação: Factory e Singleton

Desvantagens do Singleton

- Pode gerar alto acoplamento
- Difícil de testar
- Deve ser usado com cautela





Aula 26 - Padrões de Criação: Factory e Singleton



Comparação Factory x Singleton

- **Factory:** controla como objetos são criados
- **Singleton:** controla quantas instâncias podem existir
- Podem ser usados em conjunto



Aula 26 - Padrões de Criação: Factory e Singleton



Atividade Prática - Logger Singleton

Criar um Logger Singleton que registra ações do sistema

- . Método registrar(mensagem)
- . Método listar()



Aula 26 - Padrões de Criação: Factory e Singleton



Atividade Prática - Factory + Singleton

Criar fábrica de conexões que retorna sempre o mesmo objeto Singleton Exemplo: conexão com banco de dados



Aula 26 - Padrões de Criação: Factory e Singleton

Boas práticas com padrões de criação

- Factory ajuda a criar objetos de forma flexível
- Singleton garante instância única
- Logger é exemplo clássico de Singleton





Aula 26 - Padrões de Criação: Factory e Singleton



Materiais para Estudo

- Livro: Padrões de Projeto – GoF
- Real Python: <https://realpython.com/python-singleton/>
- Refactoring Guru: <https://refactoring.guru/design-patterns>
- Curso Udemy: Design Patterns em Python



Aula 26 -Padrões de Projeto e SOLID



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO